Nama: Sailendra Oriza Yudistira
NIM: 250660221002
Kelas: SI-IIIA

Soal 1

Tujuan dari encapsulation dalam OOP adalah untuk membatasi akses langsung ke data dan metode dalam sebuah objek. Dengan menggunakan encapsulation, kita dapat mengontrol bagaimana data diakses dan dimodifikasi, sehingga meningkatkan keamanan dan integritas data. Data tertentu perlu dikendalikan melalui method atau property untuk memastikan bahwa perubahan pada data tersebut dilakukan dengan cara yang benar dan sesuai dengan aturan bisnis yang berlaku. Hal ini juga membantu dalam menjaga konsistensi dan validitas data, serta memudahkan pemeliharaan kode karena perubahan pada implementasi internal tidak akan mempengaruhi kode eksternal yang menggunakan objek tersebut.

Soal 2

Fungsi `super().__init__()` pada constructor subclass digunakan untuk memanggil constructor dari superclass. Hal ini diperlukan agar subclass dapat mewarisi dan menginisialisasi atribut-atribut yang didefinisikan dalam superclass. Dengan memanggil `super().__init__()`, kita memastikan bahwa semua atribut dan logika inisialisasi yang ada di superclass dijalankan, sehingga subclass dapat memiliki semua properti dan perilaku yang diwarisi dari superclass. Tanpa pemanggilan ini, atribut-atribut dari superclass tidak akan diinisialisasi, yang dapat menyebabkan error atau perilaku yang tidak diinginkan saat menggunakan objek dari subclass.

Soal 3

Overriding adalah konsep dalam OOP di mana subclass menyediakan implementasi baru untuk metode yang sudah didefinisikan dalam superclass. Dengan overriding, subclass dapat mengubah atau menyesuaikan perilaku metode yang diwarisi dari superclass sesuai dengan kebutuhan spesifiknya. Contoh penerapan overriding pada method yang diwarisi dari superclass adalah sebagai berikut:

```
class Pengguna:
    def tampilkan_peran(self):
        return "Pengguna"
```

```
class Mahasiswa(Pengguna):
    def tampilkan_peran(self):
        return "Mahasiswa"
```

Soal 4

Inheritance sebaiknya digunakan ketika terdapat hubungan **is-a** karena konsep ini mencerminkan hubungan hierarkis antara kelas. Misalnya, jika kita memiliki kelas `Mahasiswa` yang merupakan subclass dari kelas `Pengguna`, maka kita dapat mengatakan bahwa “Mahasiswa is a Pengguna”. Dengan menggunakan inheritance, kita dapat memanfaatkan kembali kode yang ada di superclass, mengurangi duplikasi kode, dan memudahkan pemeliharaan serta pengembangan sistem. Selain itu, inheritance memungkinkan polymorphism, di mana objek dari subclass dapat digunakan di tempat yang mengharapkan objek dari superclass, sehingga meningkatkan fleksibilitas dan skalabilitas kode.

Soal 5

Output dari `Mahasiswa().tampilkan_peran()` adalah:

```
Mahasiswa
```

Karena method `tampilkan_peran()` di kelas `Mahasiswa` telah mengoverride method yang sama di kelas `Pengguna`, sehingga ketika kita memanggil method tersebut pada objek `Mahasiswa`, implementasi dari subclass yang akan dijalankan. Konsep OOP yang ditunjukkan di sini adalah **overriding**, di mana subclass dapat menyediakan implementasi spesifik untuk metode yang diwarisi dari superclass, memungkinkan perilaku yang berbeda sesuai dengan konteks objek yang digunakan.